

OptimalCover: WiFi Access Point Placement in Smart City Environments via Capacitated Facility Location

Ognean Jason-Dennis

Faculty of Mathematics and Computer Science
Babeş-Bolyai University, Cluj-Napoca, Romania
[TODO: your.email@ubbcluj.ro]

Abstract—The proliferation of smart city initiatives has intensified the demand for reliable and cost-effective public wireless infrastructure. This paper addresses the problem of optimal Wi-Fi access point (AP) placement across urban demand zones, formulated as a variant of the Capacitated Facility Location Problem (CFLP). Given a set of urban demand zones weighted by pedestrian traffic density and a set of candidate router sites generated from a uniform grid over the urban area, we seek to minimize total weighted service distance and deployment cost subject to capacity and budget constraints. We present a complete mixed-integer linear programming (MILP) formulation of the problem and describe a Greedy heuristic solution approach implemented in C++, which iteratively selects candidate sites by maximum marginal coverage gain. A Simulated Annealing metaheuristic is currently under development. Distances between candidate sites and demand zones are computed via the Haversine formula, accounting for the curvature of the Earth. The model is evaluated on real-world urban data from Cluj-Napoca, Romania, sourced from OpenStreetMap via the Overpass API, and experimental results demonstrate [TODO: brief result summary]. Our system, OPTIMALCOVER, provides an open and transparent optimization framework for municipal decision-makers planning public Wi-Fi deployments, offering reproducible, data-driven guidance for infrastructure investment decisions.

Index Terms—facility location, WiFi placement, smart city, integer programming, capacitated optimization, urban infrastructure

I. INTRODUCTION

Smart cities leverage digital infrastructure to improve urban services, sustainability, and quality of life [8]. A foundational component of any smart city ecosystem is ubiquitous, reliable wireless connectivity: without it, IoT sensors, mobile services, and real-time monitoring cannot function effectively. Municipalities worldwide are therefore investing in large-scale public WiFi deployments; however, determining *where* to place access points to maximize coverage while respecting budget constraints remains a challenging combinatorial optimization problem.

Formally, we are given a set of urban demand zones (streets, plazas, parks) weighted by estimated pedestrian traffic, and a set of candidate router sites generated from a uniform grid over the urban bounding area. The goal is to select a subset of candidate sites at which to install WiFi routers, and to assign demand zones to routers, such that total weighted service distance and deployment cost are minimized, capacity limits

are respected, and the total cost does not exceed a fixed budget. This problem is NP-hard in general, as it subsumes the classical Capacitated Facility Location Problem (CFLP) [1].

While commercial tools such as Ekahau and Cisco DNA Spaces address indoor AP placement, they operate as black boxes and do not expose the underlying optimization model. Academic work on outdoor urban WiFi placement [5] often relies on set covering formulations that do not capture capacity constraints or weighted pedestrian demand. To the best of our knowledge, no prior work applies a full capacitated FLP formulation to outdoor smart city WiFi deployment using real urban data sourced from OpenStreetMap.

Contributions. The main contributions of this paper are:

- A complete mathematical formulation of the smart city WiFi placement problem as a budget-constrained CFLP with weighted demand zones (Section III).
- A C++ implementation of a Greedy heuristic solver evaluated on real OpenStreetMap data from Cluj-Napoca, Romania (Section IV).
- Experimental analysis of the Greedy solver in terms of solution quality and runtime (Section V).

The remainder of this paper is organized as follows. Section II reviews related work. Section III presents the mathematical formulation. Section IV describes the implementation. Section V reports experimental results. Section VI concludes the paper.

II. RELATED WORK

A. Facility Location Problems

The Facility Location Problem (FLP) is a classical combinatorial optimization problem that has been extensively studied since the 1960s. The uncapacitated variant seeks to open a subset of facilities to minimize fixed opening and assignment costs [2]. Melkote and Daskin [1] extended this to a combined capacitated facility location and network design problem, showing applications in telecommunications and regional planning. An, Singh, and Svensson [3] presented the first LP relaxation with a constant integrality gap for the capacitated variant, after all prior algorithms relied solely on local search.

B. Coverage Location Problems

The Maximal Covering Location Problem (MCLP), introduced by Church and ReVelle [2], seeks to place a fixed number of facilities to maximize demand coverage within a threshold distance. With over 1,500 citations, it remains a foundational model in location analysis and has been integrated into GIS software such as ArcGIS [4]. The Set Covering Location Problem (SCLP) minimizes the number of facilities needed to cover all demand. Recent work [9] applies dynamic programming ideas to MCLP, drawing connections to the 0/1 knapsack problem.

C. WiFi Access Point Placement

Kouhbor et al. [5] apply a set covering location-allocation model to find optimal AP positions satisfying demand points under resource constraints in an outdoor campus environment. Shande et al. [6] introduce a 3D coverage model for indoor AP placement, incorporating a radio propagation model into the standard covering formulation. In the context of wireless mesh networks, [7] formulates a maximal wireless covering problem with survivability constraints using binary integer programming, applied to a real city.

D. Smart City Wireless Deployment

Abdulwahid and Mishra [8] provide a systematic mapping of deployment optimization algorithms for wireless sensor networks in smart cities (2015–2022), finding that 35% of studies enhance swarm optimization algorithms.

E. Positioning of Our Work

Unlike prior approaches, OptimalCover applies a full budget-constrained CFLP formulation to *outdoor* public WiFi deployment, uses *real pedestrian traffic data* from an urban environment, and provides a transparent, open optimization framework compared to commercial black-box tools.

III. PROBLEM FORMULATION

A. General Capacitated FLP

We first recall the classical Capacitated Facility Location Problem as a baseline. Let I be the set of demand points and J the set of candidate facility locations. Table I summarizes the parameters.

TABLE I
PARAMETERS — GENERAL CAPACITATED FLP

Symbol	Description
I	Set of demand points (clients / zones), indexed by i
J	Set of candidate facility locations, indexed by j
f_j	Fixed cost of opening facility j
c_{ij}	Cost of serving demand point i from facility j
d_i	Demand of point i
Q_j	Capacity of facility j

Decision variables: $y_j \in \{0,1\}$ equals 1 if facility j is opened; $x_{ij} \geq 0$ is the fraction of demand of i served by j .

Objective:

$$\min Z = \sum_{i \in I} \sum_{j \in J} c_{ij} x_{ij} + \sum_{j \in J} f_j y_j \quad (1)$$

Constraints:

$$\sum_{j \in J} x_{ij} = d_i, \quad \forall i \in I \quad (C1)$$

$$\sum_{i \in I} x_{ij} \leq Q_j y_j, \quad \forall j \in J \quad (C2)$$

$$x_{ij} \leq d_i y_j, \quad \forall i, j \quad (C3)$$

$$y_j \in \{0,1\}, \quad \forall j \in J \quad (C4)$$

$$x_{ij} \geq 0, \quad \forall i, j \quad (C5)$$

Constraint (C1) ensures full demand satisfaction; (C2) enforces capacity and links assignment to facility opening; (C3) prevents assignment to closed facilities; (C4) enforces binary facility decisions; (C5) ensures non-negativity.

B. Smart City WiFi Infrastructure

We now present the full formulation for the OptimalCover problem. Table II lists the sets and parameters.

TABLE II
PARAMETERS — SMART CITY WiFi FORMULATION

Symbol	Description
I	Set of demand zones, indexed by i
J	Set of candidate router sites, indexed by j
B	Maximum available budget
con_i	Number of connections required in zone i
d_{ij}	Distance from zone i to router site j
Q_j	Maximum connections router j can handle
f_j	Fixed cost of placing a router at site j
r	Maximum coverage radius ($r = 100\text{m}$)

Decision variables: $x_{ij} \in [0,1]$ is the fraction of zone i 's connections served by router j ; $y_j \in \{0,1\}$ equals 1 if a router is placed at site j .

Objective:

$$\min Z = \sum_{i=1}^{|I|} \sum_{j=1}^{|J|} \text{con}_i \cdot d_{ij} \cdot x_{ij} + \sum_{j=1}^{|J|} f_j \cdot y_j \quad (2)$$

The first term minimizes total weighted distance (connections $\times$ distance); the second minimizes total deployment cost.

Constraints:

$$\sum_{j=1}^{|J|} x_{ij} = 1, \quad \forall i \in I \quad (\text{U1})$$

$$\sum_{i=1}^{|I|} \text{con}_i \cdot x_{ij} \leq Q_j y_j, \quad \forall j \in J \quad (\text{U2})$$

$$x_{ij} \leq y_j, \quad \forall i \in I, j \in J \quad (\text{U3})$$

$$\sum_{j=1}^{|J|} f_j \cdot y_j \leq B \quad (\text{U4})$$

$$x_{ij} = 0, \quad \forall i \in I, j \in J : d_{ij} > r \quad (\text{U7})$$

$$0 \leq x_{ij} \leq 1, \quad \forall i \in I, j \in J \quad (\text{U5})$$

$$y_j \in \{0, 1\}, \quad \forall j \in J \quad (\text{U6})$$

(U1) ensures every zone is fully assigned; (U2) enforces router capacity; (U3) links zone assignment to router placement; (U4) enforces the budget; (U7) restricts assignments to candidates within the maximum coverage radius $r = 100$ m, reflecting reliable outdoor Wi-Fi signal range [5]; (U5) bounds the continuous assignment variables; (U6) enforces binary router placement.

C. Assumptions

- Distances d_{ij} are precomputed using the Haversine formula, accounting for the curvature of the Earth over urban-scale separations.
- Deployment cost f_j includes cabling and equipment.
- A zone's connections may be split across multiple routers.
- Router capacity Q_j is expressed in number of connections.
- A demand zone can only be served by a candidate router within a maximum coverage radius of $r = 100$ m, reflecting reliable outdoor Wi-Fi signal range in urban environments.

IV. IMPLEMENTATION

A. Data Collection and Preprocessing

Real-world input data for OPTIMALCOVER is sourced from OpenStreetMap (OSM) via the Overpass API. The data pipeline consists of two stages: a *fetch* stage and a *processing* stage.

1) *Fetch Stage*: The fetch stage issues a structured Overpass QL query over a bounding box corresponding to Cluj-Napoca, Romania. The query retrieves four categories of OSM way elements that represent high-footfall outdoor areas: `highway=pedestrian`, `highway=footway`, `leisure=park`, and `place=square`. The raw response is a JSON geometry file containing the node-level coordinates of each way, which is cached locally for reproducibility.

2) *Processing Stage*: The processing stage transforms raw OSM ways into the two input sets required by the CFLP formulation: demand zones I and candidate router sites J .

a) *Demand zones*.: Each OSM way element is converted into a single demand zone $i \in I$. The centroid $(\hat{\text{lat}}_i, \hat{\text{lon}}_i)$ of the zone is computed as the arithmetic mean of the coordinates of all nodes in the way's geometry. The physical length ℓ_i of the way is estimated by summing the Haversine distances between consecutive nodes:

$$\ell_i = \sum_{k=1}^{|G_i|-1} \text{Hav}(g_k, g_{k+1}), \quad (3)$$

where $G_i = (g_1, \dots, g_{|G_i|})$ is the ordered node sequence of way i and $\text{Hav}(\cdot, \cdot)$ denotes the Haversine distance in metres. The connection demand of zone i is then defined proportionally to its physical extent:

$$\text{con}_i = \lfloor \lambda \cdot \ell_i \rfloor + 1, \quad (4)$$

where $\lambda = 0.267$ connections per metre is a scaling factor calibrated to produce realistic demand loads for the study area.

b) *Candidate router sites*.: Candidate sites J are generated as a uniform grid over the bounding box of all demand zones. The grid spacing is set to $\delta = 100$ m. The angular increments in latitude and longitude corresponding to δ metres are:

$$\Delta\phi = \frac{\delta}{111,320}, \quad \Delta\lambda = \frac{\delta}{111,320 \cdot \cos(\bar{\phi})}, \quad (5)$$

where $\bar{\phi}$ is the mean latitude of the bounding box. Every grid point is assigned a uniform capacity $Q_j = 150$ connections and a uniform fixed deployment cost $f_j = 2,000$ monetary units.

c) *Distance matrix*.: The distance d_{ij} between demand zone i and candidate site j is computed using the Haversine formula, which accounts for the curvature of the Earth and is appropriate for the kilometre-scale distances encountered in an urban deployment:

$$d_{ij} = 2R \arcsin \left(\sqrt{\sin^2 \frac{\Delta\phi}{2} + \cos \phi_i \cos \phi_j \sin^2 \frac{\Delta\lambda}{2}} \right), \quad (6)$$

where $R = 6,371,000$ m is the mean Earth radius and $\Delta\phi, \Delta\lambda$ are the differences in latitude and longitude between i and j .

d) *Output format*.: The pipeline serializes the processed problem instance to a single JSON file containing the arrays `zones` and `candidates`, which is subsequently consumed by the C++ solver infrastructure.

B. Solver

We implement a *Greedy* heuristic in C++ that provides a fast, high-quality solution to the CFLP. The solver operates on a binary vector $\mathbf{y} \in \{0, 1\}^{|J|}$ indicating which candidate sites are open, together with an assignment matrix $\mathbf{x} \in [0, 1]^{|I| \times |J|}$ satisfying constraints (U1)–(U3) and (U5)–(U7) of the formulation in Section III.

The objective value of a solution $(S, \mathbf{x})$, where $S = \{j \in J : y_j = 1\}$ is the set of open sites, is:

$$Z(S) = \sum_{i \in I} \sum_{j \in S} \text{con}_i \cdot d_{ij} \cdot x_{ij} + \sum_{j \in S} f_j. \quad (7)$$

1) *Greedy Assignment*: The Greedy solver assigns each demand zone i to its nearest open candidate with sufficient remaining capacity. Zones are processed in decreasing order of con_i to reduce the risk of capacity infeasibility — large zones are served first so that high-demand areas are not left without coverage. The assignment is implemented using a `CandidateStructure` spatial index, which maintains two collections: `active` (open candidates with remaining capacity) and `inactive` (unopened candidates). For each zone, the solver first queries `active` for the nearest candidate with sufficient remaining capacity. If the active set cannot fully satisfy the zone’s demand, the solver queries `inactive` for the nearest unopened candidate, opens it if the budget allows, and inserts it into `active` with its remaining capacity. The process repeats until the zone is fully served or the problem is declared infeasible. Formally, the assignment for zone i is:

$$j^*(i) = \arg \min_{j \in S_{\text{active}}} d_{ij} \quad \text{subject to} \quad \sum_{i' \mapsto j} \text{con}_{i'} \leq Q_j, \quad (8)$$

where S_{active} is the current set of open candidates with remaining capacity. If no candidate in `active` or `inactive` within radius r can serve zone i , or if opening a new candidate would exceed the budget B , a `NoFeasibleSolutionException` is thrown. The full procedure is given in Algorithm 1.

Algorithm 1 Greedy Solver

Require: Zones I sorted by con_i descending, candidates J , costs $\{f_j\}$, capacities $\{Q_j\}$, budget B , radius r
Ensure: Open set S , assignment $\mathbf{x}$

```

1: active  $\leftarrow \emptyset$ , inactive  $\leftarrow J$ , Cost  $\leftarrow 0$ 
2: for each zone  $i \in I$  do
3:   need  $\leftarrow \text{con}_i$ 
4:   while need  $> 0$  do
5:      $c \leftarrow \text{QUERYNEAREST}(\text{active}, i, r, \text{need})$ 
6:     if  $c = \text{NONE}$  then break
7:     end if
8:      $x_{i,c.\text{id}} \mathrel{+}= c.\text{used}/\text{con}_i$ 
9:     need  $\mathrel{-}= c.\text{used}$ 
10:    UPDATE(active,  $c$ ,  $c.\text{capacity} - c.\text{used}$ )
11:   end while
12:   while need  $> 0$  do
13:      $c \leftarrow \text{QUERYNEAREST}(\text{inactive}, i, r, \text{need})$ 
14:     if  $c = \text{NONE}$  or Cost  $+ f_c > B$  then
15:       throw NoFeasibleSolutionException
16:     end if
17:      $x_{i,c.\text{id}} \mathrel{+}= c.\text{used}/\text{con}_i$ 
18:     need  $\mathrel{-}= c.\text{used}$ 
19:     Cost  $\mathrel{+}= f_c$ ,  $y_{c.\text{id}} \leftarrow \text{TRUE}$ 
20:     copy  $\leftarrow c$ 
21:     REMOVE(inactive, copy)
22:     if  $c.\text{capacity} - c.\text{used} > 0$  then
23:       copy.capacity  $\leftarrow c.\text{capacity} - c.\text{used}$ 
24:       INSERT(active, copy)
25:     end if
26:   end while
27: end for
28: return  $\mathbf{y}$ ,  $\mathbf{x}$ 
```

2) *Simulated Annealing Solver*: The Simulated Annealing solver is currently under development and will be described in a future revision of this paper.

C. System Architecture

The OPTIMALCOVER system is implemented in C++ and organized into four layers: a *data layer*, an *objective layer*, a *solver layer*, and an *output layer*.

a) *Data layer*: A JSON parser reads the `problem.json` file produced by the Python pipeline and populates a `Problem` object consisting of three components: a vector of `Zone` objects (each carrying con_i , $\hat{\text{lat}}_i$, $\hat{\text{lon}}_i$), a vector of `Candidate` objects (each carrying Q_j , f_j , and coordinates), and the precomputed Haversine distance matrix $D \in \mathbb{R}^{|I| \times |J|}$ shared across all components.

b) *Objective layer*: Evaluation of the objective function (7) is decoupled from the solvers through an abstract `Objective` base class exposing a single virtual method `evaluate(Problem, Solution)  $\rightarrow$  double`. The concrete `WifiObjective` class implements this interface for the smart city WiFi formulation, computing the total weighted distance plus deployment cost as defined in equation (7). This separation allows the objective function to

be swapped or extended — for instance, to incorporate signal propagation models — without modifying the solver logic.

c) Solver layer.: An abstract `Solver` base class holds a reference to the `Problem` instance and exposes a single virtual method `solve() → Solution`. Currently one concrete class derives from it: `GreedySolver`, which uses a `CandidateStructure` spatial index maintaining separate active and inactive collections for efficient nearest-neighbour queries within radius r . A `SimulatedAnnealingSolver` is under development. A future `BranchAndBoundSolver` will slot into the same interface without requiring changes to any other layer.

d) Output layer.: The `Solution` object — storing the binary open-set vector $\mathbf{y} \in \{0, 1\}^{|J|}$ and the assignment matrix $\mathbf{x} \in [0, 1]^{|I| \times |J|}$ — is serialized back to JSON for downstream consumption, logging, or visualization.

The class hierarchy is summarized below:

```
Objective (abstract)
  -- WifiObjective

Solver (abstract)
  -- GreedySolver           [uses CandidateStructure]
  -- SimulatedAnnealingSolver [TODO]
  -- BranchAndBoundSolver   [future]

CandidateStructure { active, inactive }
Problem { zones[], candidates[], distMatrix[][] }
Solution { y[], x[][] }
```

V. EXPERIMENTAL RESULTS

[TODO: Describe experiments after implementation is complete.]

A. Experimental Setup

[TODO: Instance sizes, hardware used, parameter values (B , Q_j , etc.).]

B. Results

[TODO: Tables and figures showing: objective value, runtime, number of routers placed, coverage achieved.]

C. Discussion

[TODO: Interpret results. What trade-offs did you observe between cost and coverage? How does the budget constraint B affect the solution?]

VI. CONCLUSION

[TODO: Write conclusion after experiments are complete.]

In this paper, we presented OPTIMALCOVER, a transparent optimization framework for public WiFi access point placement in smart city environments. We formulated the problem as a budget-constrained Capacitated Facility Location Problem and implemented a Greedy heuristic solution evaluated on real OpenStreetMap data from Cluj-Napoca, Romania. Experimental results show [TODO: main finding]. A Simulated

Annealing metaheuristic and a Branch-and-Bound solver are planned as future work.

Future work. Promising directions include implementing the Simulated Annealing metaheuristic with min-cost b -matching for optimal assignment, extending the model to dynamic demand (pedestrian traffic varying over time), and applying reinforcement learning to enable fast, approximate placement decisions without solving the full MIP at each time step.

REFERENCES

- [1] S. Melkote and M. S. Daskin, “Capacitated facility location/network design problems,” *European Journal of Operational Research*, vol. 129, no. 3, pp. 481–495, 2001.
- [2] R. Church and C. ReVelle, “The maximal covering location problem,” *Papers in Regional Science*, vol. 32, no. 1, pp. 101–118, 1974.
- [3] H.-C. An, M. Singh, and O. Svensson, “LP-based algorithms for capacitated facility location,” *arXiv preprint arXiv:1407.3263*, 2014.
- [4] A. T. Murray, “Maximal coverage location problem,” *International Regional Science Review*, vol. 39, no. 1, pp. 5–27, 2016.
- [5] L. M. Ali, O. Khanji, and E. Özceylan, “Location-allocation optimization of Wi-Fi access points at Gaziantep University campus,” in *Proc. Int. Conf. Industrial Engineering and Operations Management (IEOM)*, Manila, Philippines, Mar. 2023, pp. 2125–2136.
- [6] [TODO: Fill in full details for the ScienceDirect 2015 3D coverage paper.]
- [7] [TODO: Fill in full details for the Dublin wireless mesh network paper.]
- [8] H. M. Abdulwahid and A. Mishra, “Deployment optimization algorithms in wireless sensor networks for smart cities: A systematic mapping study,” *Sensors*, vol. 22, no. 14, p. 5094, 2022.
- [9] S. Samanta, A. R. Kalathotia, S. J. Gonchi, V. K. K. Adiraju, and S. K. Nettema, “Maximal covering location problem: A set coverage approach using dynamic programming,” *arXiv preprint arXiv:2509.23334*, 2025.